SECTOR-BASED AI ASSIGNMENT

Insurance Sector — Claims Processing Assistant

Assignment type: Loop Engineering | **Level:** Beginner to Intermediate

1. Business Scenario

An insurance company receives claims with customer details, policy numbers, incident descriptions, and supporting documents. Build a prototype assistant that checks whether required fields are present, retrieves mock policy/claim information, summarizes the case, and routes incomplete or suspicious cases to a human claims officer. This is a learning prototype using fictional data; it must not approve or reject real claims.

2. Objective

Build a Python prototype that demonstrates the seven loop patterns from the original Loop Engineering assignment. You may use an LLM for answer generation, but use fictional/mock records for claim and policy lookups. The system should prepare information for a human reviewer, not make a real insurance decision.

3. Apply the Seven Loop Patterns

Loop pattern	How to apply it
1. Retry Loop	Retry a temporary failed lookup or an incomplete draft a maximum of three times. Do not retry forever, and do not retry a failed authorization check.
2. Reflection Loop	Ask the model to review its claim summary for missing details, unclear wording, and unsupported statements, then improve it.
3. Validation Loop	Check required fields such as claim ID, policy ID, incident date, incident description, and document status. Return a clear list of missing fields.
4. Evaluation Loop	Use a separate rubric to score the summary for completeness, factual support, clarity, and whether it avoids unsupported conclusions.
5. Tool Loop	Use mock tools to retrieve policy status and claim status from a local dictionary or CSV. Pass tool results back into the workflow.
6. Human-in-the-Loop	Require a claims officer to review suspected fraud, conflicting records, missing evidence, or any proposed high-impact decision.
7. Agent Loop	Let the workflow decide whether to validate fields, look up a policy, retrieve claim details, request missing information, or prepare a summary for human review.

4. Coding Requirements

1. Create fictional policy and claim records in a dictionary or CSV.
2. Implement each loop as a separate function and explain its purpose in comments.
3. Limit retry, reflection, evaluation, and agent iterations to a maximum of three.
4. Validate required fields before calling the mock lookup tools.
5. Handle unknown claim IDs, missing fields, empty model output, and tool failures.
6. Keep a human approval step for high-risk or conflicting cases.

7. Write at least eight tests and print whether each expected outcome passed.

5. Sample Test Scenarios

Test input / scenario	Expected behavior
All required fields present; known claim ID	Retrieve mock record and prepare summary for review
Claim ID not found	Return a not-found message; do not invent a record
Incident date missing	Validation loop identifies the missing field
Temporary tool failure	Retry up to the configured limit, then fail gracefully
Conflicting policy and claim information	Flag the conflict and request human review
Possible fraud indicator	Escalate for human investigation; do not accuse or decide automatically
Empty generated summary	Validation fails and bounded recovery is attempted
Human reviewer rejects the summary	Ask for feedback and revise within the retry limit

6. Mentor Questions

1. What is the business problem, and who will use the assistant?
2. What inputs are mandatory before a claim can be reviewed?
3. How is a Retry Loop different from a Validation Loop in this project?
4. What should happen if the policy lookup tool fails three times?
5. How will you stop the assistant from inventing policy coverage or claim facts?
6. What is the difference between Reflection and Evaluation in your implementation?
7. Which cases must be escalated to a human claims officer, and why?
8. What should the assistant do when claim data and policy data conflict?
9. How will you test valid, incomplete, unknown, and suspicious claim scenarios?
10. What metrics would you report: field-validation pass rate, summary quality, tool failure rate, or escalation rate?
11. How would you deploy this safely, and what data should not be logged?
12. Why must this prototype not make the final real-world claim approval decision?

7. Expected Deliverables

Submit the notebook or Python file, a workflow diagram, fictional test data, at least eight test results, a short explanation of each loop, and a brief limitations/safety note.

8. Evaluation Rubric (100 Marks)

Evaluation criteria	Marks
Business understanding and workflow design	15
Correct use of the seven loop patterns	20
Validation and bounded error handling	20
Test cases and evaluation evidence	20
Code clarity and explanation	15
Safety, privacy, and human escalation	10
Total	100

Mentor instruction: Ask the learner to explain the reason for each loop, demonstrate at least one failure case, and show how the system avoids making unsupported decisions. Reward reasoning, safe handling, and test evidence—not only code execution.